

基于 RK3588 的夜间交通免补光暗光增强与车牌识别系统

摘要

低照度环境下，普通摄像头成像噪声大、细节丢失，车牌难以识别，一般需要闪光灯补光，给夜间驾驶的安全性带来考验。本作品基于飞凌 ELF2（瑞芯微 RK3588）嵌入式开发板，设计并实现了一套完全运行在边缘端的系统，实现暗光图像实时增强和车牌识别。系统以 ZWO ASI585MCRAW 相机为输入，采用长短交替曝光连续采集三帧 RAW 图像，送入自研的 NBINet 多帧融合去噪网络（该网络在自建的 3953 组低照度数据集上训练），在 RK3588 内置 NPU 上以 FP16 精度实时推理，输出清晰的 RGB 图像；去噪结果一路经 DRM/KMS 直接显示到 MIPI 屏实时预览，一路送入 HyperLPR3 两阶段车牌识别，识别出的车牌号、置信度、抓拍图写入本地，并通过板载 WiFi 提供的轻量 HTTP 服务供手机浏览器实时查询。整套推理流水线以多线程 C 语言实现（采集、去噪、显示、识别四线程 + 环形缓冲），端到端约 8fps，插电即用。实测中系统可将暗光噪声图像增强为清晰彩色图，并正确识别车牌。手机端可按车牌归组查询抓拍照片与命中帧数。本作品完整覆盖 RAW 采集、NPU 去噪、车牌识别、显示与查询的端侧闭环，具有实时、低功耗、可离线部署的特点。

关键词：暗光图像增强；RAW 域多帧融合去噪；车牌识别；边缘 AI；端侧推理；RK3588 NPU；

第一部分 作品概述

1.1 功能与特性

1. 暗光实时增强：采用 RAW 域长短三帧融合去噪，将低照度噪声图像增强为清晰彩色图像，保留暗部细节与真实色彩。

2. 车牌检测与识别：对增强后图像进行车牌定位（YOLO）与字符识别

(CTC)，支持中国蓝牌/黄牌等，输出车牌号与置信度。

3. 实时显示：去噪结果经 DRM/KMS 直接输出到 MIPI DSI 屏，画面无撕裂。

4. 手机查询：板载 WiFi 提供 HTTP 服务，手机浏览器即可查询识别记录，按车牌自动归组、显示抓拍图与命中帧数，支持车牌号搜索。

5. 离线运行：全流程运行于 RK3588 边缘端，无需云端与 PC，上电自动运行。

1.2 应用领域

本作品属于赛题“选题方向二：端侧 AI 视觉应用”（智能安防方向），面向夜间及低照度场景的车辆管理与安防需求。典型应用包括：智慧停车场出入口车牌识别；小区、园区门禁车辆放行；夜间道路卡口、违停抓拍；执法记录仪与车载巡逻终端；无人值守站点的车辆通行记录。本系统以算法增强替代物理补光，且边缘端离线运行，适合供电与网络受限的野外或移动场景。

1.3 主要技术特点

1. RAW 域多帧融合去噪：直接处理传感器 12 bit RAW，长-短-长三帧，融合出高信噪比图像，优于普通 RGB 视频后处理。

2. NPU FP16 端侧加速：NBINet 经 PyTorch 训练→ONNX→RKNN 转换后部署于 RK3588 NPU。实测发现 INT8 量化会破坏去噪模型的颜色重建，最终采用 FP16，在精度与速度间取得平衡。

3. NPU+CPU 异构并行：去噪模型运行于 NPU，车牌识别在 CPU 上推理，在不同线程并行执行，NPU 去噪与 CPU 识别同时进行、互不争抢，充分利用 RK3588 的异构算力。

4. C 语言实现多线程实时流水线：采集、去噪、显示、识别四线程，以环形缓冲解耦，去噪与识别各自消费全部帧，互不阻塞。

5. 两阶段车牌识别：HyperLPR3 的 YOLO 检测和 CTC 序列识别，ncnn 推理。

6. 显示与查询：DRM/KMS 直控 MIPI 屏；无第三方依赖的 HTTP 服务，手机免装 APP 即可查询。

1.4 主要性能指标

如表 1.1 所示。

指标	参数
去噪模型	基于 D2HNet 改造，INT8 量化
去噪分辨率	1080P
去噪速度	10 fps
车牌识别耗时	80ms 内
显示	1080P
端到端帧率	10fps
联网查询	未涉及

表 1.1

1.5 主要创新点

1. 算法层面：

(1) RAW 域短-长-短三帧融合去噪：去噪网络 NBINet 借鉴 D2HNet 思路，直接在 12-bit RAW 域端到端完成多帧对齐、融合与增强，以两帧短曝光作为运动“锚点”保留清晰边缘、长曝光帧补足暗部亮度，绕过传统 ISP，从底层同时抑制噪声与运动拖影。

(2) 将原网络中 NPU 不支持或低效的算子替换为 NPU 友好算子，做 NPU 友好化改造，在几乎不损失效果的前提下，使一个面向 GPU 的研究网络得以在 RK3588 NPU 上原生高效加速。

2. 工程部署层面：

(1) NPU+CPU 异构并行流水线：去噪运行于 NPU、车牌识别经 ncnn 运行于 CPU，双环形缓冲、并行执行，每帧兼顾显示与识别。

(2) 边缘离线运行：板载 WiFi +轻量 HTTP 服务支持手机按车牌归组查询。

1.6 设计流程

如图 1.1 所示。

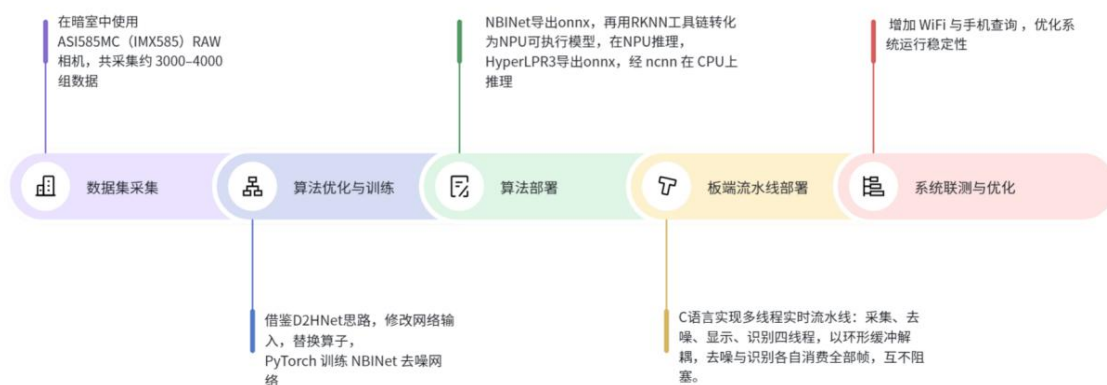


图 1.1

第二部分 系统组成及功能说明

2.1 整体介绍

数据流如图 2.1 所示。

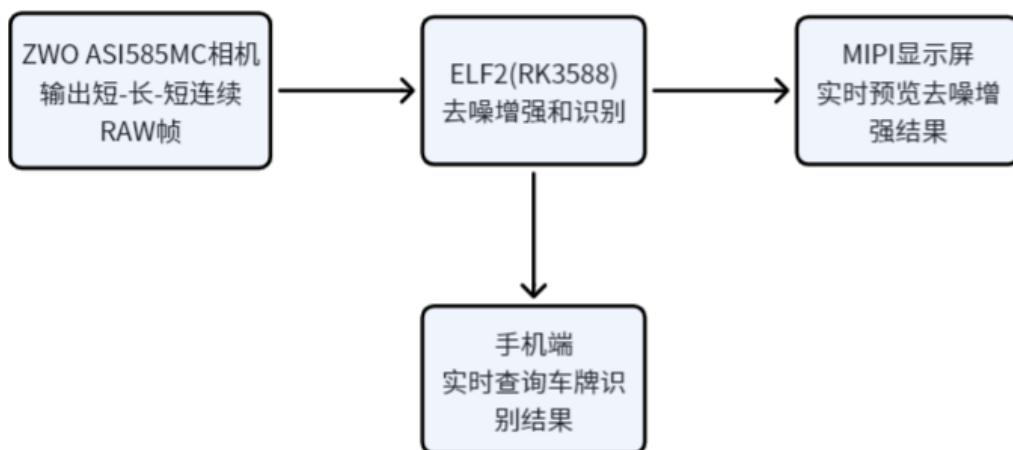


图 2.1

2.2 硬件系统介绍

硬件由 ELF2 (RK3588) 核心板、ZWO ASI585MC USB 相机、MIPI DSI

显示屏（1024×600）、板载 CF-AX200 WiFi/蓝牙模块构成。相机经 USB 接入，MIPI DSI 输出至显示屏，WiFi 提供手机查询网络。连接如下图 2.2 所示。

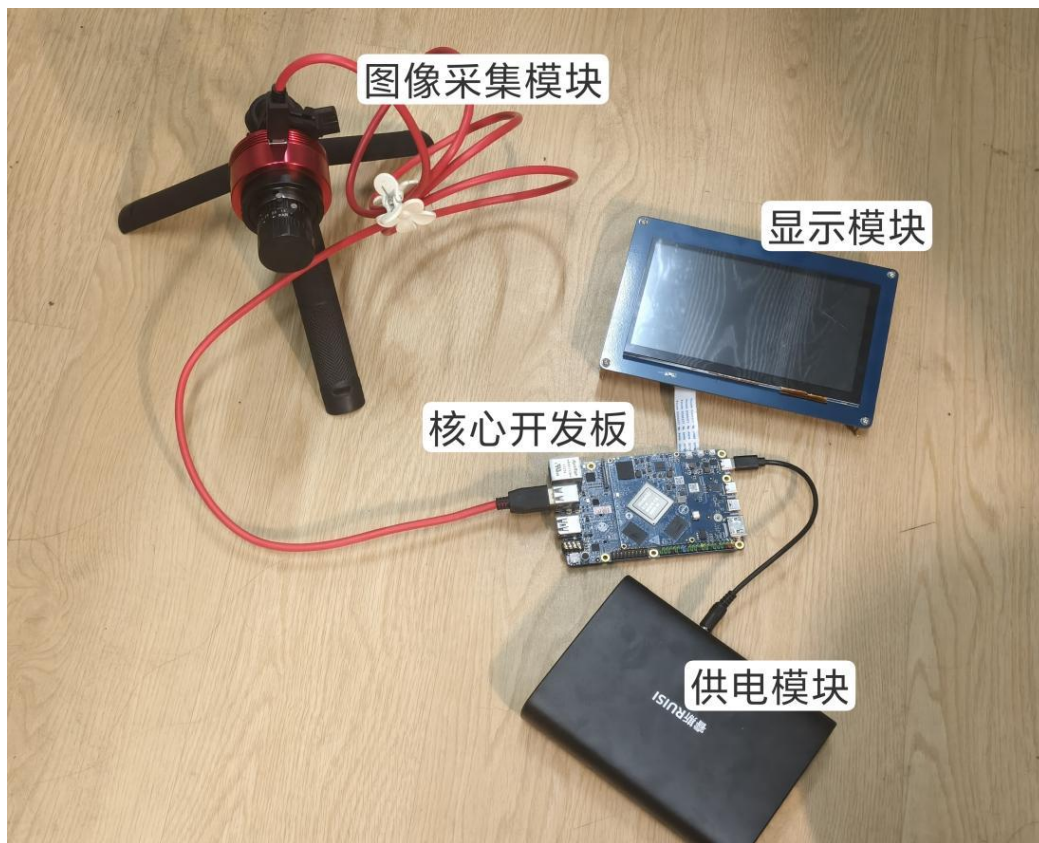


图 2.2

2.3 软件系统介绍

2.3.1 软件整体介绍

软件整体架构如下图 2.3 所示。

本系统软件采用“采集-去噪-显示-识别-服务”一体化的多线程流水线架构，围绕端侧实时视觉处理进行设计。

系统从 USB 相机采集夜间低照度 RAW 图像，经过采集模块完成 Bayer 拆分与曝光序列组织后，由去噪模块在 NPU 上完成多帧融合增强，再分别送入显示模块和识别模块并行处理，其中显示模块负责将增强结果实时输出到 MIPI 屏，识别模块在 CPU 上完成车牌检测、字符识别与结果存储，Web 服务模块则进一步将识别结果以网页形式对外提供查询。各模块之间通过环形缓冲解耦，实现了计算与 I/O 并行，兼顾

了实时性、稳定性和可维护性。

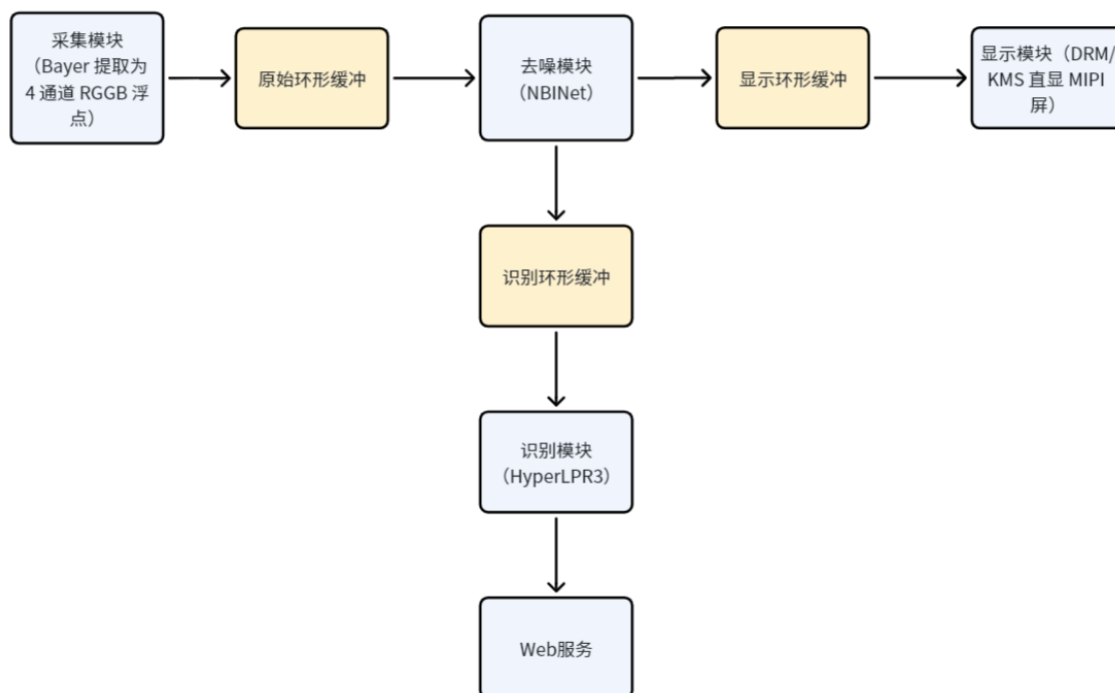


图 2.3

2.3.2 软件各模块介绍

1. 采集模块：

采集模块由 `capture_thread` 主循环驱动，首先调用 `asi_init` 完成相机初始化、分辨率和硬件 ROI 配置，再根据项目训练输入尺寸将输出固定在 960×544 的 RAW16 Bayer 图像。该模块的核心目标是把传感器数据直接整理为模型可用的四通道 RAW 特征，避免在前端引入额外 ISP 处理导致分布偏移。初始化完成后，模块进入交替曝光工作模式，曝光时序为短曝光、长曝光、短曝光循环，即 S-L-S。每次曝光切换时会丢弃一帧过渡帧，以减少自动寄存器稳定前对后续去噪的干扰。

在数据处理链路上，采集线程每得到一帧原始数据后，都会分配 `raw_buf_t` 缓冲并调用 `bayer_extract` 做 Bayer 拆分与归一化。其流程是：读取 2×2 Bayer 子块、执行黑电平和白电平裁剪、按 Bayer 排列映射到 R/Gr/Gb/B 四平面，并将像素归一化到 $0 \sim 1$ 。

$$v' = \frac{\min(\max(v, 96), 4095) - 96}{4095 - 96}$$

输出变量包括 `data`（四平面 float 数组）、`ts_us`（时间戳）、`exposure`（短或长曝光标识）和 `frame_id`（帧序号），并通过 `raw_ring` 送入下游。

2. 去噪模块：

去噪模块是整个系统的核心计算单元，其任务不是简单地对单帧图像做增强，而是基于短、长曝光多帧信息，在 RAW 域完成噪声抑制、细节恢复和去马赛克重建。与传统 D2HNet 不同，本项目的 NBINet 已经针对 RK3588 NPU 的部署约束做了重构：输入形式从原始的 RGB 域长-短帧，改成了 RAW 域的短-长短三帧输入结构；网络内部也不再使用 DWT/IWT、DCN 等 NPU 兼容性差的算子，而是改用普通卷积、残差块和 PixelShuffle 等更适合端侧加速的结构。因此，虽然整体思路仍然借鉴 D2HNet 的双分支、多帧融合框架，但实现上已经不是原始 D2HNet，而是面向实时部署重新设计后的轻量化网络。

从数据流上看，去噪模块接收来自采集模块的三帧 RAW 数据，按短、长、短的时序组织成网络输入。其中两帧短曝光被拼接为 8 通道 `short_cat`，一帧长曝光作为 4 通道 `long_raw`，这种设计的目的是同时保留短曝光在高亮区域的细节信息和长曝光在暗部区域的信噪比优势。输入张量均采用 RGGB 四平面表示，并在采集端已经完成黑电平校正和归一化，因此网络侧不再进行额外的颜色空间转换。

输入特征表示为：

$$F_s = [S_1, S_2] \in \mathbb{R}^{8 \times H \times W}, \quad F_l = L \in \mathbb{R}^{4 \times H \times W}$$

其中 S_1, S_2 分别表示第一、二帧短曝光 RAW 特征，形状均为 $4 \times H \times W$ 。 L 表示长曝光 RAW 特征。

输出为全分辨率 RGB 图像：

$$Y \in \mathbb{R}^{3 \times 2H \times 2W}$$

网络结构上，NBINet 采用双编码器、单融合瓶颈、三级解码器的架构。短曝光分支负责从短帧拼接输入中提取对高亮和运动更稳健的结构信息，而长曝光分支负责提取暗部纹理和低照信号；两个分支在多个尺度上逐层下采样提取特征，最终在瓶颈层进行 `concat` 融合，再通过 1×1 卷积和残差块完成跨时域信息整合。解码阶段则使用 `PixelShuffle` 逐级上采样，并在每一级与对应尺度的短帧、长帧特征进行 `skip` 连接，从而兼顾细节保真和重建稳定性。最后，网络通过残差输出形式将高分辨率重建分量与长帧参考分量相加，得到最终 RGB 结果。这样的设计使网络既保留了 D2HNet 的多帧融合思想，又显著降低了算子复杂度，更适合在 NPU 上落地。

本项目针对端侧部署对算子进行了比较彻底的改造。原始 D2HNet 中常见的小波分解/重建、变形卷积以及较复杂的对齐模块，对 NPU 来说通常意味着较差的图优化支持和较高的调度开销，因此本项目没有沿用这些结构，而是用标准 `Conv2d`、残差块、特征拼接和 `PixelShuffle` 来替代。这样做的一个直接好处是模型可以更容易地导出为 ONNX 并进一步转换为 RKNN，同时在 RK3588 上获得更稳定的推理性能。

该模块由 `npu_thread` 执行，模型加载后首先通过 `rknn_query` 读取输入输出张量格式，再将采集模块送来的三帧 RAW 数据打包成 `short_cat` 和 `long_raw` 两路输入。推理完成后，输出张量被转换成 `float RGB` 图像，并根据现场安装方向做必要翻转校正，再分别投递到显示环和识别环。由于该模块位于全系统的中间层，既要保证算法质量，又要控制时延，因此其设计目标不是单纯提高 PSNR，而是在可接受画质下尽量压缩端到端延迟并避免资源争用。模块关键输入包括 `short_cat`、`long_raw`、模型路径和翻转参数，关键输出包括 `rgb->data`、`frame_id` 以及推理时延统计 `npu_ms`。

3. 显示模块

显示模块由 `display_thread` 驱动，采用 DRM/KMS 直显路径而不是传统 `framebuffer` 写屏，核心原因是直显能在嵌入式桌面服务停用后仍保持稳

定扫描输出。线程启动后先停止 Weston，打开 card0 获取 DRM master，自动选择已连接的 connector 和可用 CRTC，创建 dumb buffer 并通过 `drmModeSetCrtc` 绑定到显示管线，建立一块可持续刷新的扫描内存。图像渲染部分由 `convert_to_fb` 完成，流程是从 `rgb_ring` 取出一帧、按目标屏幕分辨率进行双线性插值、执行亮度风格增强后写入 XRGB8888。插值本质是四邻域加权，可表示为：

$$f(x, y) = (1 - \alpha)(1 - \beta)Q_{11} + \alpha(1 - \beta)Q_{21} + (1 - \alpha)\beta Q_{12} + \alpha\beta Q_{22}$$

其中 α, β 是亚像素偏移。为改善夜景“发灰”观感，模块还做了对比度与饱和度增强，增益参数由 CONTRAST 与 SAT 控制（后部署时确定值分别为 1.1 和 1.3）。该模块关键输入变量是 `out_fmt`、`dst_stride`、`mode` 分辨率和 `rgb` 数据，关键输出是显存扫描缓冲中的最终像素流。它不参与算法决策，但决定用户对系统的主观感受。

4. 识别模块：

由 `lpr_thread` 完成，运行在 CPU 上并基于 `ncnn`，按“检测-识别-分类”三级链路处理去噪结果。对识别模块进行部署时，起初我们尝试将算法部署在 NPU 上，实测发现可能会和去噪算法抢占 NPU，从而导致丢帧，另外还发现该算法在 NPU 的推理速度为 120ms/帧（如图 2.4 所示），远慢于在 CPU 的 40–60 ms/帧，推测可能对于小模型来说，NPU 侧计算负载不足以

完全抵消 DMA 数据搬运和任务调度开销有关。

```

root@elf2-buildroot:/mnt/sdcard# /tmp/rknn_bench /mnt/sdcard/y5fu_320x_sim.rknn
Model: /mnt/sdcard/y5fu_320x_sim.rknn
Inputs: 1, Outputs: 1
in[0]: dims=[1,320,320,3] size=614400 type=1 fmt=1 scale=1.000000 zp=0
out[0]: dims=[1,6300,15,0] size=189000 type=1 fmt=3
Results (5 warmup + 20 loops):
  min: 38.94 ms (25.7 FPS)
  avg: 44.68 ms (22.4 FPS)
  max: 48.91 ms (20.4 FPS)
root@elf2-buildroot:/mnt/sdcard# ^C
root@elf2-buildroot:/mnt/sdcard# /tmp/rknn_bench /mnt/sdcard/y5fu_640x_sim.rknn
Model: /mnt/sdcard/y5fu_640x_sim.rknn
Inputs: 1, Outputs: 1
in[0]: dims=[1,640,640,3] size=2457600 type=1 fmt=1 scale=1.000000 zp=0
out[0]: dims=[1,25200,15,0] size=756000 type=1 fmt=3
Results (5 warmup + 20 loops):
  min: 77.90 ms (12.8 FPS)
  avg: 83.01 ms (12.0 FPS)
  max: 96.12 ms (10.4 FPS)
rknn@elf2-buildroot:/mnt/sdcard# /tmp/rknn_bench /mnt/sdcard/rpv3_mdct_160_r3.
Model: /mnt/sdcard/rpv3_mdct_160_r3.rknn
Inputs: 1, Outputs: 1
in[0]: dims=[1,48,160,3] size=46080 type=1 fmt=1 scale=1.000000 zp=0
out[0]: dims=[1,20,78,0] size=3120 type=1 fmt=3
Results (5 warmup + 20 loops):
  min: 9.33 ms (107.2 FPS)
  avg: 11.97 ms (83.6 FPS)
  max: 14.75 ms (67.8 FPS)
rknn@elf2-buildroot:/mnt/sdcard# /tmp/rknn_bench /mnt/sdcard/litemodel_cls_96x_r
Model: /mnt/sdcard/litemodel_cls_96x_r1.rknn
Inputs: 1, Outputs: 1
in[0]: dims=[1,96,96,3] size=55296 type=1 fmt=1 scale=1.000000 zp=0
out[0]: dims=[1,3,0,0] size=6 type=1 fmt=3
Results (5 warmup + 20 loops):
  min: 0.90 ms (1109.9 FPS)
  avg: 1.05 ms (953.1 FPS)
  max: 1.23 ms (813.0 FPS)

```

图 2.4

线程启动后加载三个模型：YOLO 检测模型、CTC 序列识别模型和车牌颜色类别模型。主循环从 `lpr_ring` 取帧后先缩放到 `320×320` 进行 YOLO 推理，再通过 `yolo_decode_raw` 解码候选框并执行 `nms_boxes` 去重。NMS 的核心判据是 IoU，定义为：

$$IoU = \frac{|B_i \cap B_j|}{|B_i \cup B_j|}$$

当 IoU 超过阈值时抑制低分框，保证同一车牌不被重复输出。

对保留框，模块将坐标映射回原图，裁剪 ROI 后送入 CTC 模型。CTC 解码由 `ctc_decode` 完成，规则是逐时刻取最大类别、去除 blank、压缩连续重复字符，得到最终 `plate_str`。对应的分类分支把同一 ROI 缩放到 `96×96`，输出牌型类别。识别结果会附带 `frame_id`、`ts_us`、`conf`、`type` 等字段写入 `plates.txt`，并在满足阈值与时间门控条件时生成抓拍图。该模块关键输入是 `rgb->data` 与三模型路径，关键输出是 `plate_str`、`plate_type`、`conf`、`img_name` 和 `lpr_ms`。它是系统业务价值的核心承载模块，直接决定查车牌能力和误识率。

5. Web 服务模块

Web 服务模块的作用是把板端离线生成的识别结果转化为可直接在手机或浏览器中访问的查询界面，它本质上是整个系统的结果出口。该模块没有引入额外的 Web 框架，而是直接基于 socket 实现一个轻量级 HTTP 服务器，原因在于板端资源有限，同时系统只需要满足结果查询、图片查看、记录归组这几类固定功能，不需要复杂的会话管理、数据库和路由框架。这样的实现方式结构简单、依赖少、启动快，也便于和主识别流程解耦部署。

lpr_server 启动后先监听指定端口，随后不断接受客户端连接。每次收到请求后，服务端先读取原始 HTTP 报文，再从请求行中解析出方法和路径，根据路径进行分发：访问根路径时返回 HTML 页面，访问 /api/plates 时返回 JSON 格式的结构化数据，访问 /files/xxx 时返回图片或文本文件。整个过程遵循最基本的请求-响应模型，因此实现上不依赖额外中间件，也没有引入动态网页模板引擎。对于本项目而言，这种方式足以支撑手机端实时查看识别记录，而且维护成本很低。

数据的核心来源是 plates.txt。识别模块每识别到一条结果，就会把帧号、时间戳、车牌字符串、车牌类型、置信度以及对应抓拍图片名写入该文本文件。Web 服务模块在处理 /api/plates 请求时，会重新读取 plates.txt，并按 plate 字段对记录进行归组。归组时不仅统计每个车牌的命中次数，还会保留最高置信度和最近出现顺序，同时把对应的抓拍图像记录到 shots 列表中。这样前端看到的不是单条离散日志，而是按车辆聚合后的结果卡片，更符合实际使用场景，比如同一辆车多次经过时，用户可以直接看到它的所有抓拍记录、出现频次和最佳识别结果。

这个归组过程本质上是一个日志解析到哈希式聚合再到时间排序的流程。服务端逐行解析 plates.txt，每一行先提取 plate、frame、ts、conf、img 等字段，再判断该车牌是否已经存在于当前分组集合中。如果不存在，就新建一组；如果存在，则更新命中计数和最高置信度，并把新的抓拍项追加到该组的 shots 中。为了控制内存增长，每个车牌只保留最近若干张抓拍图，避免文本和图片数量持续增加后带来不必要的内存压力。这个设计的好处是结果

查看始终保持最近状态优先，更适合现场快速检索。

网页前端本身也采用了轻量设计。页面加载后会通过浏览器定时轮询 /api/plates 接口，周期性拉取最新数据并刷新列表，因此不需要 WebSocket 或长连接推送。前端页面提供车牌搜索框，用户输入省份简称、车牌片段或完整车牌号后，就会在本地对已获取数据进行过滤，快速定位目标车辆。每个车牌卡片中会展示命中次数、最高置信度以及多张抓拍缩略图，缩略图点击后可直接打开原图查看。这样设计的原理是把“结果展示”和“结果传输”分离：传输只负责把结构化数据送到浏览器，展示则完全由前端静态页面完成，逻辑清晰且维护简单。

第三部分 完成情况及性能参数

3.1 整体介绍

系统已完成整体联调，实现从相机采集到手机查询的端侧闭环，上电自动运行。如图 3.1 所示。



图 3.1

3.2 工程成果

3.2.1 机械成果：如 3.1 节所示。

3.2.2 电路成果：本项目使用开发板，未自主进行电路图绘制。

3.2.3 软件成果：

(1) 屏幕实时显示去噪后清晰彩色画面，白天以及夜间成像效果如图 3.2 所示；



图 3.2

(2) 手机浏览器访问板端服务，可见按车牌归组的识别记录、抓拍图廊与搜索框。如图 3.3，3.4 所示（出于隐私考虑，车牌部分打码处理）。



图 3.3



图 3.4

3.3 特性成果

对照赛题选题方向二的考核指标，本作品实现结果如下：

1. 图像质量

在低照度条件下，将高噪声暗光图像增强为清晰彩色图像暗部细节与真实色彩得到恢复，色彩还原准确；在验证集上达到 PSNR 35.0 dB、SSIM 0.924（模型训练结果如图 3.5 所示），重建质量与结构相似度均较高，PC 端算法处理后的图像与输入以及 GT 图对照如图 3.6，3.7 所示。

```
val: 410 | epoch: 199
val: epoch: 199, psnr: 35.000, ssim: 0.924
```

图 3.5



图 3.6



图 3.7 (图 3.6、3.7 数据集中原有 S-L-S RAW 图像经过基础冲洗)

板端暗室环境实拍以及处理后效果如图 3.8 所示。



图 3.8

2. 视觉算法性能

车牌检测 + 识别端到端约 8fps; 实测正确识别车牌, 去噪约 120 ms/帧、识别约 40 - 60 ms/帧, 测试时终端输出如图 3.9 所示 (出于隐私考虑, 车牌部分打码处理)。

```

[STAT] frames=48 drops=0 npu=118.6ms lpr=49.7ms
[NPU] frame 49: 151.03 ms
[LPR] YOLO out: w=15 h=6300 c=1 total=94500
[LPR] grid=6300 max_conf=0.7188@6064 n>=0.35=5
[LPR] best box: xywh=143.50,218.62,182.50,105.38 conf=0.7188
[LPR] frame=48 plate=苏A6 F8 type=0 conf=0.72 lpr=47.8ms
[NPU] frame 50: 100.89 ms
[LPR] YOLO out: w=15 h=6300 c=1 total=94500
[LPR] grid=6300 max_conf=0.7417@5868 n>=0.35=6
[LPR] best box: xywh=139.75,220.50,180.88,110.44 conf=0.7417
[LPR] frame=49 plate=苏A6 F8 type=0 conf=0.74 lpr=50.8ms
[NPU] frame 51: 144.03 ms
[LPR] YOLO out: w=15 h=6300 c=1 total=94500
[LPR] grid=6300 max_conf=0.8042@5868 n>=0.35=5
[LPR] best box: xywh=138.00,220.50,178.50,108.50 conf=0.8042
[LPR] frame=50 plate=苏A6 F8 type=0 conf=0.80 lpr=58.7ms
[NPU] frame 52: 115.55 ms
[LPR] YOLO out: w=15 h=6300 c=1 total=94500
[LPR] grid=6300 max_conf=0.7300@5868 n>=0.35=6
[LPR] best box: xywh=140.50,216.38,178.88,108.69 conf=0.7300
  
```

图 3.9

3. 数据集构建

团队自主采集并构建了含 3953 组样本的低照度 RAW 数据集用于模型训练，采用源视频合成 + 暗室屏幕重拍的受控采集流程。首先从高分辨率视频中抽取片段，并使用 RIFEm 插帧生成约 960fps 的高帧率时间轴；在该时间轴上选定中间时刻作为 GT，对其前后的短曝光时刻生成 S1、S2，同时围绕 GT 取 40/60/80 帧进行平均，合成带运动模糊的长曝光图 L。随后将 S1、L、S2、GT 依次在显示器上静态播放，在暗室中使用 ZWO ASI585MC 相机固定机位拍摄 RAW 数据。其中 GT 画面会连续采集 200 帧 RAW 并平均，得到低噪声 GT_mid。每次切换显示内容后设置等待时间，并在曝光或增益变化后丢弃若干帧，以减少显示残留和相机设置切换带来的干扰。最终每个样本包含时间对应的 S1、L、S2 和 GT_mid RAW 帧，用于构建三帧输入的低光照增强、去噪和运动恢复实验数据。拍摄流程图 3.10 所示。

整合公开街景视频资源，通过插帧与时序采样技术，构建适配 0.01–0.1lux 极低照度、500m 远距离场景的高质量训练数据。



图 3.10

4. 系统鲁棒性

识别在车牌远近、角度变化下自适应，对倒装/镜像等成像方向问题已在流水线中校正；识别置信度低于阈值时不误报。

5. 系统稳定性：四线程流水线长时间运行无丢帧（drops=0），采集通道稳定，数据写入与手机查询稳定。

6. 手机查询：识别记录按车牌自动归组，显示全部抓拍图、命中帧数与置信度，支持车牌号搜索，图见 3.2.3 节。

3.4 实测指标（参考 1.4 节中预期指标）

如表 3.1 所示。

指标	参数
主控平台	瑞芯微 RK3588（4×A76+4×A55，6 TOPS NPU）
图像传感器	ZWO ASI585MC（Sony IMX585 12-bit RAW）
去噪模型	NBNet，三帧 RAW 融合，FP16
去噪分辨率	输入 272×480 输出 544×960 RGB (考虑推理速度，算法本身没有分辨率要求)

去噪精度	验证集 PSNR 35.0 dB, SSIM 0.924
去噪速度	约 120 ms/帧 (约 8 fps, NPU)
车牌识别耗时	约 40–60 ms/帧 (ncnn, CPU ARM NEON)
显示	MIPI DSI 1024×600, DRM/KMS 直显
端到端帧率	约 8 fps, 实测无丢帧
联网查询	板载 WiFi + HTTP 服务, 手机浏览器

表 3.1

第四部分 总结

4.1 可扩展之处

1. 修复 RK3588 USB3 驱动后，可提升相机带宽与帧率，支持更高分辨率；
2. 采用滑窗帧复用（将上一组的后短曝光帧复用为下一组的前短曝光帧），由“3 帧新帧出 1 帧”变为“2 帧新帧出 1 帧”，在不增加算力的前提下提升有效帧率；
3. 支持多路相机与多车牌并行识别；
4. 识别结果可上云或推送，形成车辆通行数据库；
5. 采用混合量化（敏感层 FP16 + 其余 INT8）进一步提速；

4.2 心得体会

本作品的最大收获，是完整走通了从训练模型到边缘端部署的工程链路，并在过程中解决了大量真实问题。

比如相机在 USB3 口持续采集时频繁复位、无法出图，我们通过对比最简单帧采集和持续流采集，USB3 和 USB2，以及搜索同类问题，定位到是 RK3588 的 xHCI 控制器在 SuperSpeed 持续流下的稳定性问题，最终使用 USB2 通道，同时进行相机硬件 ROI 裁剪（降低带宽的同时正好能取到模型所需区域）避开，把帧率从 2fps 提升到 8fps。这让我们体会到边缘部署中，先用最小实验隔离问题、再对症下药，比盲目改代码高效得多。

再比如，实践让我们认识到模型量化并非无损搬运。最初 INT8 量化后的去噪结果出现严重偏色，我们通过逐环节 dump 中间数据、在 PC 上逐层

比对，确认是 INT8 量化破坏了去噪网络对通道细微差异的重建并使上采样层失效、分辨率减半，改用 FP16 后颜色与分辨率均恢复正常。

另外，端到端系统里每一层的约定都务必要对齐。调试时车牌识别一度输出乱码，排查后发现是多个问题叠加：相机成像 180° 倒装导致字符方向错误、YOLO 输入归一化误除 255 使图像接近全黑、CTC 时间步数写错导致读越界产生固定乱码尾巴。逐一修正后才正确识别出车牌。在工程部署过程中无论是预处理、通道顺序、张量形状等什么“接口约定”，任何一处不一致都会让整条链路失效，必须用数据验证而非想当然。

在本次项目实践中，我们锻炼了从算法、系统到硬件的全栈调试能力，也让我们认识到，一个能实际运行的边缘 AI 系统，其价值不仅在于模型本身，更在于把每一层可靠地拼接起来。

第五部分 参考文献

- [1] ZHAO Y, XU Y, YAN Q, et al. D2HNet: joint denoising and deblurring with hierarchical network for robust night image restoration[C]//European Conference on Computer Vision (ECCV). Cham: Springer, 2022.
- [2] C. Chen, Q. Chen, J. Xu, V. Koltun. Learning to see in the dark[C]//2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition. Salt Lake City, UT, USA: IEEE, 2018: 3291-3300. DOI: 10.1109/CVPR.2018.00347.
- [3] Hai Jiang, Binhao Guan, Zhen Liu, Xiaohong Liu, Jian Yu, Zheng Liu, Songchen Han, Shuaicheng Liu. Learning to see in the extremely dark [C] // Proceedings of the IEEE/CVF International Conference on Computer Vision. Paris, France: IEEE, 2025.
- [4] Hasinoff S W, Sharlet D, Geiss R, Adams A, Barron J T, Kainz F, Chen J, Levoy M. Burst photography for high dynamic range and low-light imaging on mobile cameras[J]. ACM Transactions on Graphics, 2016, 35(6): 192:1-192:12. DOI: 10.1145/2980179.2980254.
- [5] Zhang ,Dan,Zhao , et al.Dual-constraint burst image denoising method[J].Frontiers of Information Technology & Electronic Engineering,2022,23(2):1-14.DOI:10.1631/FITEE.2000353.
- GRAVES A, FERNÁNDEZ S, GOMEZ F, et al. Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks[C]//Proceedings of the 23rd International Conference on Machine Learning (ICML). New York: ACM, 2006: 369-376.
- [7] WEI K, FU Y, YANG J, et al. A physics-based noise formation

- model for extreme low-light raw denoising[C]//IEEE/CVF. 2020
IEEE/CVF Conference on Computer Vision and Pattern Recognition
(CVPR). Piscataway: IEEE, 2020: 2755-2764.
- [8] JOCHER G. YOLOv5[EB/OL]. (2020)[2026-07-07].
<https://github.com/ultralytics/yolov5>.
- [9] HyperLPR: 高性能中文车牌识别框架[EB/OL]. [2026-07-07].
<https://github.com/szad670401/HyperLPR>.
- [10] 腾讯. ncnn: 高性能神经网络前向计算框架[EB/OL]. [2026-07-07].
<https://github.com/Tencent/ncnn>.
- [11] Rockchip. RKNN-Toolkit2[EB/OL]. [2026-07-07].
<https://github.com/airockchip/rknn-toolkit2>.
- [12] 瑞芯微电子股份有限公司. RK3588 技术参考手册[Z]. 福州: 瑞芯微电子, 2023.
- [13] Sony Semiconductor Solutions. IMX585 CMOS image sensor datasheet[Z]. Tokyo: Sony, 2022.
- [14] 保定飞凌嵌入式技术有限公司. ELF2 (RK3588) 开发板快速使用手册[Z]. 保定: 飞凌嵌入式, 2024.